Bayesian Machine Learning in Quantitative Finance

Chapter 13: A Bayesian Investment Analyst on the JSE

Based on Mongwe, Mbuvha & Marwala

Chapter Outline

Notation You'll Need / Recap I

This chapter is an *application* chapter: it takes the Bayesian logistic regression model (Ch. 2/6) and the MALA / HMC / S2HMC samplers (Ch. 3, 7, 9, 11) that you already know in detail, and uses all three *together, on the same model and the same data*, to build and compare a "Bayesian investment analyst" that reads sell-side analyst reports on JSE-listed stocks and predicts whether the report's price call will turn out to be directionally correct.

Recap: Bayesian Logistic Regression (Ch. 2/6)

For a binary label $y \in \{0, 1\}$ and feature vector $\mathbf{x} \in \mathbb{R}^d$, the likelihood is Bernoulli with success probability $\sigma(\mathbf{w}^\top \mathbf{x})$, where $\sigma(z) = 1/(1 + e^{-z})$ is the logistic sigmoid. Placing a prior on $\mathbf{w}$ and combining with the likelihood via Bayes' theorem gives a posterior $p(\mathbf{w} \mid \mathrm{D})$ that has *no closed form* once you multiply a non-Gaussian likelihood by even a Gaussian prior — hence the need for MCMC.

Notation You'll Need / Recap II

Recap: MALA / HMC / S2HMC (Ch. 3, 7, 9, 11)

All three are *gradient-informed* MCMC methods that use $\nabla_{\mathbf{w}} \ln p(\mathbf{w} \mid \mathrm{D})$ to propose better-than-random-walk moves:

- **MALA** – one step of discretised Langevin diffusion (drift = gradient, diffusion = Brownian noise) + a Metropolis correction.
- **HMC** – simulates Hamiltonian dynamics with a leapfrog integrator over L steps, then Metropolis-corrects for the discretisation error; MALA is the special case $L = 1$.
- **S2HMC** – runs leapfrog on a *shadow* (modified) Hamiltonian that the integrator conserves more accurately, raising the acceptance rate and reducing autocorrelation relative to HMC.

Notation You'll Need / Recap III

What's New in This Chapter: Two Things

1. **The features are analyst-report-derived, not asset returns.** Each "data point" is one sell-side report on one JSE stock, described by features like its target price, rating, volatility context, and past accuracy record (Sect. 13.4.1) — not a raw price series. The label is *bidirectional accuracy*: did the stock actually move in the direction the report implied?
2. **MALA, HMC and S2HMC are run side by side and compared**, not just used once to fit a model. The chapter trains the *same* Bayesian logistic regression model three times (once per sampler) on four datasets (full JSE universe + 3 sectors) and compares them on (a) predictive AUC, (b) effective sample size (ESS) and ESS normalised by execution time, and (c) which features each sampler's posterior ranks as most important.

Notation You'll Need / Recap IV

How Report Features Feed the Model

Every sell-side report is converted into a numeric feature vector $\mathbf{x}_i$ (target price, rating, volatility measures, past-accuracy scores, . . . – 14 features in total, Sect. 13.4.1). The model then places an **automatic relevance determination (ARD) prior** over the logistic regression weight vector $\mathbf{w}$, so that after training, the *posterior variance* of each weight automatically tells you how important the corresponding report feature was for predicting bidirectional accuracy – this is what makes the model "explainable" to an analyst, not just predictive.

13.1 Introduction: Why Model Sell-Side Analyst Reports? I

The Business Problem

Sell-side analysts publish reports with a **target price**, an **earnings forecast**, and a **rating** (Strong Sell / Sell / Hold / Buy / Strong Buy). Institutional investors (pension funds, asset managers) use these reports for portfolio decisions, so the *accuracy* of the reports matters a great deal.

- **Point accuracy**: how close is the target price to the actual realised price?
- **Directional (bidirectional) accuracy**: did the stock move in the *direction* the report implied? Many traders act on direction, not on the exact target level — this is the chapter's focus.

Known Issues with Analyst Reports

- Conflicts of interest: analysts may align forecasts with their own economic incentives, or herd toward a consensus with other analysts.
- Coverage is skewed toward blue-chip stocks, with mid-/small-caps under-covered.

These issues motivate combining *many* analysts' reports with a data-driven model rather than trusting any single report.

13.1 Introduction: Why Model Sell-Side Analyst Reports? II

From "AI Analyst" to "Bayesian Analyst"

Sidogi et al. (2022) proposed an **"AI analyst"**: use random forests and deep neural networks to fuse sell-side report features into directional forecasts for FTSE/JSE All Share Index companies, with little human involvement. **This chapter extends that idea into the Bayesian framework**: the same directional-forecast task, but with

- a **Bayesian logistic regression with ARD prior** (BLR-ARD) instead of a black-box ensemble/deep net, and
- **MCMC** (MALA, HMC, S2HMC) instead of point-estimate training, so that predictions come with *quantified uncertainty* and features come with an automatically learned *importance ranking*.

13.1 Introduction: Why Model Sell-Side Analyst Reports? III

Why Bayesian?

The Bayesian framework answers two questions the frequentist "AI analyst" cannot answer directly:

1. **How confident is the model?** – the posterior over $\mathbf{w}$ gives a full predictive distribution, not just a point probability.
2. **Which features actually matter?** – the ARD prior's per-feature variance hyperparameters α_i give a principled, automatically-learned feature-importance ranking (no separate feature-selection step needed).

Headline Finding (previewed)

The BLR-ARD model **outperforms on industrial and resource-sector stocks**, with **financial stocks** showing the lowest bidirectional accuracy. **Volatility-related features, plus the initial and target price**, are consistently important drivers of directional predictability across sectors (details in Sect. 13.5).

13.2 Setting Up the Model: Intuition

Intuition

We want
$\mathbb{P}$(stock moves as the report implied | report features).
Logistic regression is the natural choice: it maps a linear combination of report features through a sigmoid to get a probability in $[0, 1]$. Being *Bayesian* about $\mathbf{w}$ means we don't just get one "best" probability – we get a whole *distribution* over plausible probabilities, reflecting how much the (finite, noisy) report history really tells us.

Report features $\mathbf{x}_i$ (target price, rating, volatility, ...) $\longrightarrow$ linear score $\mathbf{w}^\top \mathbf{x}_i$ $\longrightarrow$ sigmoid $\longrightarrow$ $\mathbb{P}(y_i{=}1)$

13.2 The Likelihood (Eq. 13.1)

Negative Log-Likelihood

$$l(\mathrm{D} \mid \mathbf{w}) = \sum_{i}^{N} y_i \log(\mathbf{w}^\top x_i) + (1 - y_i) \log(1 - \mathbf{w}^\top x_i)$$

$\mathrm{D} = \{\mathbf{x}, y\}$	the data: report feature vectors $\mathbf{x}$ and bidirectional-accuracy labels y
N	number of observations (analyst reports)
$\mathbf{w}$	the parameters (weights) to be estimated

As printed in the book (Marwala et al., 2023; Mongwe et al., 2021c), the term $\mathbf{w}^\top x_i$ here plays the role of the predicted probability $p_i = \sigma(\mathbf{w}^\top x_i)$ under the usual logistic link – i.e. this is the standard Bernoulli cross-entropy log-likelihood $\sum_i y_i \log p_i + (1 - y_i) \log(1 - p_i)$ with $p_i = \sigma(\mathbf{w}^\top x_i)$; we use this explicit sigmoid form on the next slide to derive the gradient.

13.2 The Posterior and the ARD Prior (Eq. 13.2)

Un-normalised Log Posterior

$$\ln p(\mathbf{w} \mid \mathrm{D}) = l(\mathrm{D} \mid \mathbf{w}) + \ln p(\mathbf{w} \mid \alpha) + \ln q(\alpha)$$

$\ln p(\mathbf{w} \mid \alpha)$	log prior on weights given hyperparameters α
$\ln q(\alpha)$	log distribution of the hyperparameters α

Automatic Relevance Determination (ARD)

Each weight w_j has its **own** Gaussian prior, zero mean, variance α_j: $w_j \mid \alpha_j \sim \mathcal{N}(0, \alpha_j)$, with $\alpha_j \sim \mathrm{LogNormal}(0, 1)$.

- **Large** $\alpha_j \Rightarrow w_j$ can be large $\Rightarrow$ feature j is *important*.
- **Small** $\alpha_j \Rightarrow w_j$ is shrunk toward $0 \Rightarrow$ feature j is *irrelevant*.

Both $\mathbf{w}$ and α are **jointly sampled** by MCMC in this chapter (doubling the parameter space vs. Gibbs-sampling α separately, but giving more stable exploration, Mongwe et al., 2021c).

13.2 Deriving the Gradient, Step by Step (Toy 2-Feature Case) I

Why We Need This

Both MALA (Eq. 13.3) and HMC (Eq. 13.4) need $\nabla_{\mathbf{w}} \ln p(\mathbf{w} \mid \mathrm{D})$. We derive it here explicitly for a toy case with just **two features** x_{i1}, x_{i2} (e.g. "sentiment score" and "EPS revision"), so $\mathbf{w} = (w_1, w_2)$ and $p_i = \sigma(w_1 x_{i1} + w_2 x_{i2})$.

Step 1: Write the Log-Likelihood in Sigmoid Form

$$l(\mathrm{D} \mid \mathbf{w}) = \sum_{i=1}^{N} \Big[y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big], \qquad p_i = \sigma(w_1 x_{i1} + w_2 x_{i2})$$

13.2 Deriving the Gradient, Step by Step (Toy 2-Feature Case) II

Step 2: Differentiate the Sigmoid

Using the standard identity $\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$ and the chain rule, for $j = 1, 2$:

$$\frac{\partial p_i}{\partial w_j} = p_i(1 - p_i)\, x_{ij}$$

13.2 Deriving the Gradient, Step by Step (Toy 2-Feature Case) III

Step 3: Differentiate Each Term of the Log-Likelihood

$$\frac{\partial}{\partial w_j}\big[y_i \log p_i\big] = \frac{y_i}{p_i} \cdot p_i(1-p_i)x_{ij} = y_i(1-p_i)x_{ij}$$

$$\frac{\partial}{\partial w_j}\big[(1-y_i)\log(1-p_i)\big] = \frac{-(1-y_i)}{1-p_i} \cdot p_i(1-p_i)x_{ij} = -(1-y_i)p_i\,x_{ij}$$

Step 4: Add the Two Pieces — They Telescope

$$y_i(1-p_i)x_{ij} - (1-y_i)p_i\,x_{ij} = \big[y_i - y_i p_i - p_i + y_i p_i\big]x_{ij} = (y_i - p_i)\,x_{ij}$$

So the gradient of the log-likelihood, summed over reports, is remarkably simple:

$$\boxed{\frac{\partial l(\mathrm{D} \mid \mathbf{w})}{\partial w_j} = \sum_{i=1}^{N}(y_i - p_i)\,x_{ij}}$$

Step 5: Add the Gradient of the ARD Log-Prior

$\ln p(\mathbf{w} \mid \alpha) = -\frac{1}{2} \sum_j w_j^2/\alpha_j + \text{const}$, so

$$\frac{\partial \ln p(\mathbf{w} \mid \alpha)}{\partial w_j} = -\frac{w_j}{\alpha_j}$$

Step 6: The Full Gradient, for Our Toy 2-Feature Case

$$\frac{\partial \ln p(\mathbf{w} \mid \mathrm{D})}{\partial w_1} = \sum_{i=1}^{N} (y_i - p_i)\, x_{i1} - \frac{w_1}{\alpha_1}, \qquad \frac{\partial \ln p(\mathbf{w} \mid \mathrm{D})}{\partial w_2} = \sum_{i=1}^{N} (y_i - p_i)\, x_{i2} - \frac{w_2}{\alpha_2}$$

In vector form, this is exactly $f'(\mathbf{w})$ in MALA's Eq. 13.3 and $-\partial H(\mathbf{w}, \mathbf{p})/\partial \mathbf{w}$ in HMC's Eq. 13.4:

$$\nabla_{\mathbf{w}} \ln p(\mathbf{w} \mid \mathrm{D}) = \mathbf{x}^{\top}(\mathbf{y} - \mathbf{p}_{\text{prob}}) - \mathbf{w} \oslash \boldsymbol{\alpha}$$

where $\oslash$ is element-wise division and $\mathbf{p}_{\text{prob}} = (p_1, \ldots, p_N)$ stacks the predicted probabilities. This same formula, extended from 2 to 14 features, is what all three samplers use in the real JSE experiments.

13.2 Toy Running Example: 5 Fictional Analyst Reports

Intuition

To make the model concrete, we build a tiny toy "sell-side report" dataset by hand: 5 fictional reports, each with a conviction-strength feature vector, and a binary label for whether the call was bidirectionally correct. This is **entirely fictional** – a pedagogical stand-in for the book's real 29,935-report Bloomberg dataset (Sect. 13.4.1).

Report (fictional)	Rating	|sentiment|	|EPS rev.|	|target-price chg.|	y (correct?)
Alpha Capital	Buy	0.80	0.060	0.120	1
Beta Securities	Sell	0.60	0.040	0.080	1
Gamma Research	Hold	0.10	0.005	0.010	0
Delta Equities	Buy	0.50	0.030	0.070	1
Epsilon Analytics	Hold	0.20	0.005	0.015	0

Toy story: reports with *stronger conviction* (larger-magnitude signals, either bullish or bearish) tend to be vindicated more often than wishy-washy "Hold" calls with near-zero conviction – so all three features are conviction *magnitudes*, and the ARD prior gets to decide how much each one matters.

Python: BLR-ARD Log Posterior and Gradient

```python
import numpy as np

# Toy dataset: 5 fictional analyst reports (bias + 3 features)
X_raw = np.array([[0.80, 0.060, 0.120],   # Alpha Capital  (Buy)
                  [0.60, 0.040, 0.080],   # Beta Securities (Sell)
                  [0.10, 0.005, 0.010],   # Gamma Research (Hold)
                  [0.50, 0.030, 0.070],   # Delta Equities (Buy)
                  [0.20, 0.005, 0.015]])  # Epsilon Analytics (Hold)
y = np.array([1, 1, 0, 1, 0])
X = np.hstack([np.ones((5, 1)), X_raw])       # prepend bias column
alpha = np.array([4.0, 4.0, 4.0, 4.0])        # fixed ARD prior variances (toy)

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def log_posterior(w):
    """ln p(w|D): Bernoulli log-lik (Eq. 13.1) + Gaussian ARD log-prior."""
    p = sigmoid(X @ w)
    ll = np.sum(y * np.log(p) + (1 - y) * np.log(1 - p))
    log_prior = -0.5 * np.sum(w**2 / alpha)
    return ll + log_prior

def grad_log_posterior(w):
    """Analytic gradient derived step-by-step above: sum_i (y_i-p_i)x_i - w/alpha."""
    p = sigmoid(X @ w)
    return X.T @ (y - p) - w / alpha
```

Listing 1: Exact scaled log-posterior and analytic gradient (Eqs. 13.1–13.2 and the derivation above).

13.3 Three Samplers, One Posterior

Intuition

All three samplers target the *same* un-normalised posterior $p(\mathbf{w} \mid \mathrm{D})$ from Eq. 13.2 – they differ only in *how* they propose moves through $\mathbf{w}$-space. The chapter's central experiment is not "which sampler do we pick?" but **"how do MALA, HMC, and S2HMC compare on the exact same BLR-ARD model and the exact same JSE data?"** (Sect. 13.4-13.5).

	MALA	HMC	S2HMC
Gradient order used	1st	1st	1st (via shadow Ham.)
Trajectory length L	1 (implicit)	tunable (15 in this chapter)	tunable (15 in this chapter)
Target accept. rate	60%	80%	80%
Relative auto-correlation	highest	lower	lowest

13.3.1 MALA – Efficient Recap

Langevin Dynamics (Eq. 13.3)

$$d\mathbf{w}_t = \tfrac{1}{2} f'(\mathbf{w})\, dt + dZ_t$$

$\mathbf{w}$	the parameters
$f'(\mathbf{w})$	first derivative of the target $f(\mathbf{w}) = \ln p(\mathbf{w} \mid \mathrm{D})$ w.r.t. $\mathbf{w}$
Z_t	Brownian motion at fictitious time t

- Discretised via Euler-Maruyama $\Rightarrow$ a Gaussian proposal centred at $\mathbf{w} + \frac{1}{2}\epsilon f'(\mathbf{w})$; a Metropolis correction fixes the resulting detailed-balance violation.
- Converges faster than plain Metropolis-Hastings because it uses first-order gradient information – but samples are **still highly correlated** (it is, after all, a single-leapfrog-step special case of HMC).

This chapter's specifics: step size tuned via dual averaging (Hoffman & Gelman, 2014) to a target acceptance rate of 60%.

Python: MALA Sampler for the Toy Analyst Model

```python
def run_mala(n_iter, step, dim, log_posterior, grad_log_posterior, rng):
    w = np.zeros(dim)
    lp, grad = log_posterior(w), grad_log_posterior(w)
    chain = np.zeros((n_iter, dim))
    for m in range(n_iter):
        mean_fwd = w + 0.5 * step * grad                    # Langevin drift
        prop = mean_fwd + np.sqrt(step) * rng.standard_normal(dim)
        lp_prop = log_posterior(prop)
        grad_prop = grad_log_posterior(prop)
        mean_bwd = prop + 0.5 * step * grad_prop
        # Metropolis correction for the asymmetric Gaussian proposal
        log_q_fwd = -np.sum((prop - mean_fwd)**2) / (2 * step)
        log_q_bwd = -np.sum((w - mean_bwd)**2) / (2 * step)
        log_accept = (lp_prop + log_q_bwd) - (lp + log_q_fwd)
        if np.log(rng.uniform()) < log_accept:
            w, lp, grad = prop, lp_prop, grad_prop          # accept
        chain[m] = w
    return chain
```

Listing 2: From-scratch MALA (Eq. 13.3 + Metropolis correction) on the toy BLR-ARD posterior

13.3.2 HMC – Efficient Recap

Hamiltonian Dynamics (Eq. 13.4)

$$\frac{d\mathbf{w}}{dt} = \frac{\partial H(\mathbf{w}, \mathbf{p})}{\partial \mathbf{p}}, \qquad \frac{d\mathbf{p}}{dt} = -\frac{\partial H(\mathbf{w}, \mathbf{p})}{\partial \mathbf{w}}$$

$\mathbf{p}$	auxiliary momentum, same dimension as $\mathbf{w}$
$H(\mathbf{w}, \mathbf{p})$	Hamiltonian = potential (target posterior) + kinetic (Gaussian) energy

Leapfrog + Metropolis Correction (Eqs. 13.5-13.6)

Leapfrog integrates the dynamics for L steps of size ϵ; then

$$\mathbb{P}(\text{accept } \mathbf{w}^*) = \min\left(1, \frac{\exp(-H(\mathbf{w}^*, \mathbf{p}^*))}{\exp(-H(\mathbf{w}, \mathbf{p}))}\right)$$

corrects for the $\mathcal{O}(\epsilon^2)$ discretisation error. Setting $L = 1$ recovers MALA exactly.

This chapter's specifics: step size ϵ tuned by dual averaging to an 80% target acceptance rate; trajectory length **fixed at** $L = 15$ for all four experiments.

Python: HMC Sampler for the Toy Analyst Model

```python
def run_hmc(n_iter, eps, L, dim, log_posterior, grad_log_posterior, rng):
    w = np.zeros(dim)
    chain = np.zeros((n_iter, dim))
    for m in range(n_iter):
        p0 = rng.standard_normal(dim)                    # draw momentum
        w_new, p = w.copy(), p0.copy()
        grad = grad_log_posterior(w_new)
        for _ in range(L):                               # leapfrog, Eq. 13.5
            p = p + 0.5 * eps * grad
            w_new = w_new + eps * p
            grad = grad_log_posterior(w_new)
            p = p + 0.5 * eps * grad
        H0 = -log_posterior(w) + 0.5 * np.sum(p0**2)       # potential + kinetic
        H1 = -log_posterior(w_new) + 0.5 * np.sum(p**2)
        if np.log(rng.uniform()) < (H0 - H1):              # Eq. 13.6
            w = w_new
        chain[m] = w
    return chain
```

Listing 3: From-scratch HMC (leapfrog, Eq. 13.5, + acceptance, Eq. 13.6) on the toy posterior

13.3.3 S2HMC – Efficient Recap I

The Problem HMC Still Has

Leapfrog only conserves H to $\mathcal{O}(\epsilon^2)$, so H still drifts along a trajectory – causing rejections in the Metropolis step. Fix: integrate a **shadow Hamiltonian** $\tilde{H}$ that leapfrog conserves far more accurately.

Shadow Hamiltonian Expansion (Eqs. 13.7-13.9)

$$\tilde{H} = H + \epsilon H_2 + \epsilon^2 H_3 + \epsilon^3 H_4 + \ldots, \qquad \tilde{H}^{[k]} = \tilde{H} + \mathcal{O}(\epsilon^k)$$

A 4th-order truncation for leapfrog is

$$\tilde{H}^{[4]} = \mathrm{U}(\mathbf{w}) + \mathrm{K}(\mathbf{p}) + \frac{\epsilon^2}{12}\mathrm{K}_{\mathbf{p}}^{\top}\mathrm{U}_{\mathbf{ww}}\mathrm{K}_{\mathbf{p}} - \frac{\epsilon^2}{24}\mathrm{U}_{\mathbf{w}}^{\top}\mathrm{K}_{\mathbf{pp}}\mathrm{U}_{\mathbf{w}} + \mathcal{O}(\epsilon^4)$$

but this is **non-separable** in $(\mathbf{w}, \mathbf{p})$ – expensive to tune.

13.3.3 S2HMC – Efficient Recap II

Making It Separable: S2HMC (Eq. 13.10)

Sweet et al. (2009) pre-process $(\mathbf{w}, \mathbf{p})$ through the leapfrog integrator to get a **separable** shadow Hamiltonian:

$$\tilde{H}(\mathbf{w}, \mathbf{p}) = \mathrm{U}(\mathbf{w}) + \mathrm{K}(\mathbf{p}) + \frac{\epsilon^2}{24} \mathrm{U}_{\mathbf{w}}^{\top} \mathbf{M}^{-1} \mathrm{U}_{\mathbf{w}} + \mathcal{O}(\epsilon^4)$$

$\mathrm{U}(\mathbf{w})$	potential energy $= -\ln p(\mathbf{w} \mid \mathrm{D})$
$\mathrm{K}(\mathbf{p})$	kinetic energy $= \frac{1}{2} \mathbf{p}^{\top} \mathbf{M}^{-1} \mathbf{p}$
$\mathrm{U}_{\mathbf{w}}, \mathrm{U}_{\mathbf{ww}}$	Jacobian / Hessian of U; $\mathrm{K}_{\mathbf{p}}, \mathrm{K}_{\mathbf{pp}}$ analogous for K

13.3.3 S2HMC – Efficient Recap III

Reversible Pre-/Post-Processing (Eqs. 13.11-13.14)

Before leapfrog, map $(\mathbf{w}, \mathbf{p}) \mapsto (\hat{\mathbf{w}}, \hat{\mathbf{p}})$ by fixed-point iteration:

$$\hat{\mathbf{p}} = \mathbf{p} - \frac{\epsilon}{24}\left[\mathrm{U}_{\mathbf{w}}(\mathbf{w}+\epsilon\mathbf{M}^{-1}\hat{\mathbf{p}}) - \mathrm{U}_{\mathbf{w}}(\mathbf{w}-\epsilon\mathbf{M}^{-1}\hat{\mathbf{p}})\right], \quad \hat{\mathbf{w}} = \mathbf{w} + \frac{\epsilon^2\mathbf{M}^{-1}}{24}\left[\mathrm{U}_{\mathbf{w}}(\mathbf{w}+\epsilon\mathbf{M}^{-1}\hat{\mathbf{p}}) + \mathrm{U}_{\mathbf{w}}(\mathbf{w}-\epsilon\mathbf{M}^{-1}\hat{\mathbf{p}})\right]$$

Run ordinary leapfrog on $(\hat{\mathbf{w}}, \hat{\mathbf{p}})$, then **reverse** the mapping as a post-processing step. S2HMC **degenerates to HMC** when the shadow equals the true Hamiltonian.

Payoff

Because $\tilde{H}$ is conserved more accurately than H, the Metropolis step (using $\tilde{H}$, with importance weights applied afterward to correct back to the true posterior) accepts more often at a given step size ϵ $\Rightarrow$ **narrower confidence bands on ESS** and typically the **highest raw ESS** of the three samplers – at the cost of extra gradient evaluations per iteration (the fixed-point maps).

Python: S2HMC Sampler for the Toy Analyst Model

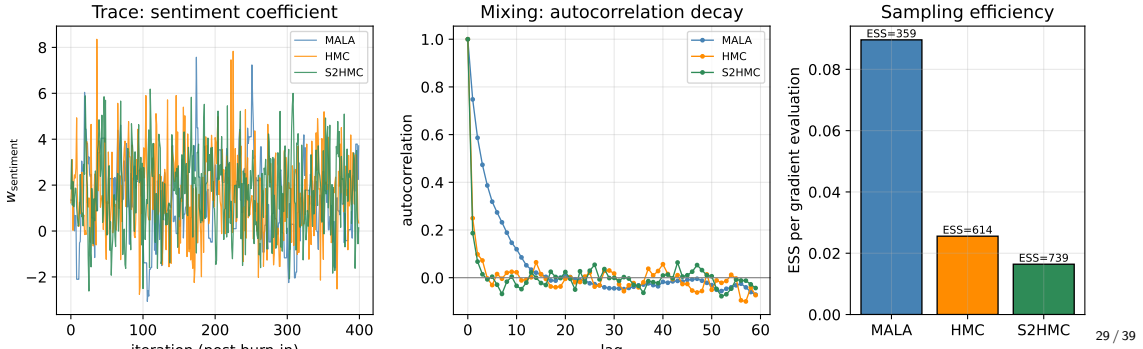
```python
def shadow_forward_map(w, p, eps, U_grad, n_fp=3):
    """Fixed-point solve for (w_hat, p_hat), Eqs. 13.11-13.12."""
    p_hat = p.copy()
    for _ in range(n_fp):
        p_hat = p - (eps/24)*(U_grad(w+eps*p_hat) - U_grad(w-eps*p_hat))
    w_hat = w + (eps**2/24)*(U_grad(w+eps*p_hat) + U_grad(w-eps*p_hat))
    return w_hat, p_hat

def shadow_backward_map(w_hat, p_hat, eps, U_grad):
    """Reverse mapping, Eqs. 13.13-13.14."""
    w = w_hat - (eps**2/24)*(U_grad(w_hat+eps*p_hat) + U_grad(w_hat-eps*p_hat))
    p = p_hat + (eps/24)*(U_grad(w_hat+eps*p_hat) - U_grad(w_hat-eps*p_hat))
    return w, p

def shadow_hamiltonian(w, p, eps, U, U_grad):
    """4th-order separable shadow Hamiltonian, Eq. 13.10 (M = I)."""
    Uw = U_grad(w)
    return U(w) + 0.5*np.sum(p**2) + (eps**2/24)*np.sum(Uw**2)
# ... leapfrog is then run on (w_hat, p_hat) exactly as in the HMC code,
# and accept/reject uses shadow_hamiltonian in place of H(w,p).
```

Listing 4: From-scratch S2HMC: shadow pre-/post-processing (Eqs. 13.11-13.14) around a leapfrog core

Toy Comparison: MALA vs HMC vs S2HMC Mixing

Intuition

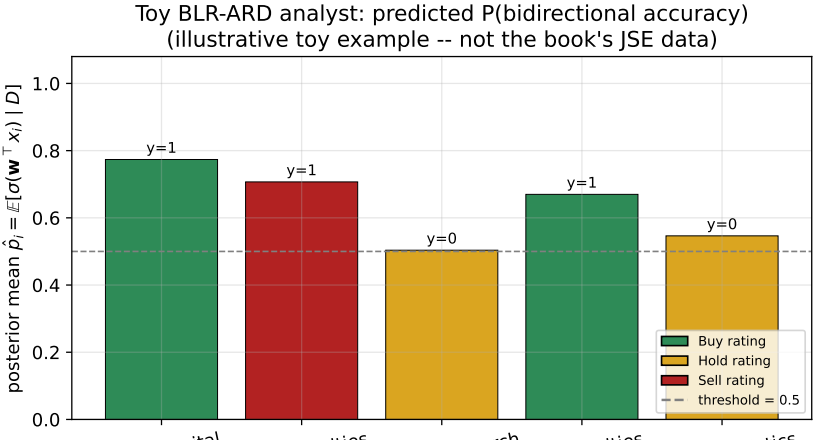
We run all three from-scratch samplers on the toy BLR-ARD posterior (5 fictional reports, Sect. 13.2) and compare how quickly each one de-correlates.



Toy Comparison: Predicted Bidirectional Accuracy

Intuition

Fitting the toy BLR-ARD model via MALA (post burn-in posterior mean of $\mathbf{w}$) lets us predict $\hat{p}_i = \mathbb{E}[\sigma(\mathbf{w}^\top x_i) \mid \mathrm{D}]$ for each fictional report.



13.4.1 Data Description

The Real JSE Dataset

Sell-side analyst reports on **FTSE/JSE All Share Index** companies, sourced from **Bloomberg**, **11 Nov 2005 – 29 Jun 2018**: **29,935 reports** in total. Target: **bidirectional accuracy** of the initial-to-realised-closing-price move over a **12-month** horizon. Class balance: **56% positive / 44% negative**.

14 features per report:

1. Firm (report issuer)
2. Target Price
3. Rating (1=Strong Sell,. . . ,5=Strong Buy)
4. P_0 (initial price)
5. TP/P_0
6. 20-day rolling volatility
7. Top40 cross-sectional volatility (CSV)
8. RelVol (rolling vol / Top40 CSV)
9. DAS (directional accuracy score)
10. MES, MAS (met-end / met-any score)
11. PAS (point accuracy score), PTM (price-target momentum), AC (analyst coverage)

13.4.2 Experiment Description

Setup

- **80/20 train-test split**; features min-max scaled.
- **2,500 posterior samples** generated per run, of which **1,000 are burn-in** (used for step-size tuning).
- Step size tuned by **primal-dual averaging** (Hoffman & Gelman, 2014) targeting **60% acceptance for MALA**, **80% for HMC and S2HMC**; trajectory length of HMC/S2HMC **fixed at** $L = 15$.

Table 13.1: The Four Experiments

Experiment	Description	# Observations
1	Full dataset	29,935
2	Resource stocks only	10,522
3	Financial stocks only	7,193
4	Industrial stocks only	12,220

13.4.3 Performance Metrics

Sampler Diagnostics

- **Trace plots** of the (un-normalised) target posterior.
- **Multivariate effective sample size (ESS)** (Vats et al., 2019) – and ESS **normalised by execution time**, since a method that costs $10\times$ more per sample needs $10\times$ the raw ESS just to break even.

Predictive Metric

ROC: true positive rate vs. false positive rate; AUC = area under the ROC

AUC is preferred here because the relative cost of false positives vs. false negatives is *unknown* for the bidirectional-accuracy task – exactly the situation where AUC is the natural summary metric.

13.5 Predictive Performance (Table 13.2)

AUC on Unseen Data (bold = best per experiment)

Algorithm	Exp. 1	Exp. 2	Exp. 3	Exp. 4
MALA	0.6181	0.6325	0.6154	**0.6698**
HMC	**0.6182**	**0.6327**	0.6158	0.6686
S2HMC	**0.6182**	0.6324	**0.6164**	0.6687

- AUCs are **very close across all three samplers** within each experiment – all three converge to essentially the same posterior predictive distribution, as they should.
- **Industrial stocks (Exp. 4)** give the highest AUC (≈ 0.67); **financial stocks (Exp. 3)** give the lowest (≈ 0.615-0.616), *worse than the full dataset* – financials are harder to call directionally.
- **Resource stocks (Exp. 2)** also outperform the full-dataset baseline.

13.5 Sampling Performance: ESS Across Methods

Qualitative Findings (Figs. 13.1-13.3)

- **MALA and HMC** give **similar** raw ESS, but with **wide confidence bands** across the four experiments.
- **S2HMC** gives **narrower confidence bands** – a direct consequence of the shadow Hamiltonian being better conserved by the pre-processed leapfrog integrator.
- On a **time-normalised** basis, **S2HMC's ESS/time is much lower** than MALA's or HMC's, because its extra pre-/post-processing steps make each iteration far more expensive – it buys better mixing *per sample*, not *per second*.

Diagnostics

All three methods show **flat negative-log-likelihood trace plots** after burn-in across all four experiments – a visual confirmation of convergence for every sampler/experiment combination.

13.5 Which Features Matter? (Tables 13.3-13.5)

Broad Agreement Across Samplers

The three MCMC methods broadly **agree** on which features matter most per sector, even though their numeric importance rankings are not identical:

- **Full JSE universe (Exp. 1):** *relative volatility* (RelVol) and *initial price* (P_0) are the most consistently important features.
- **Resource stocks (Exp. 2):** *20-day rolling volatility* and *target price* dominate.
- **Financial stocks (Exp. 3):** *analyst coverage*, *initial price*, and *target price* matter most.
- **Industrial stocks (Exp. 4):** *target price*, *initial price*, *analyst coverage*, and *Top40 cross-sectional volatility* are essential.

Takeaway for Stakeholders

Volatility context and price-level features consistently beat the raw rating/sentiment fields – and which exact feature dominates **differs by sector**, showing that a one-size-fits-all feature set is suboptimal for institutional investors screening sell-side reports.

13.6 Conclusion I

What This Chapter Did

Introduced a **Bayesian investment analyst**: a **Bayesian logistic regression with ARD prior** (BLR-ARD) that predicts the **bidirectional accuracy** of sell-side analyst reports on the JSE, trained with **MALA, HMC, and S2HMC** and compared across all three.

Key Results

- All three MCMC methods give **similar predictive AUC** – the choice of sampler does not change *what* the model learns, only *how efficiently* it explores the posterior.
- The model **outperforms on industrial and resource stocks**, with **financial stocks** the hardest sector to call.
- The BLR-ARD's automatic feature ranking is a genuine practical payoff: institutional investors get a *sector-specific* guide to which report features to focus on when challenging fund managers' strategies.

13.6 Conclusion II

Sampler Trade-off, Restated

- **MALA**: cheapest per iteration, most auto-correlated.
- **HMC**: similar ESS to MALA but with a longer, gradient-guided trajectory ($L = 15$); good general-purpose middle ground.
- **S2HMC**: best raw ESS and narrowest ESS confidence bands, but the highest per-iteration cost – worst ESS/time of the three here.

There is **no free lunch**: better mixing per sample does not automatically mean better mixing per unit of wall-clock time.

13.6 Conclusion III

Future Directions (as stated in the book)

- Replace BLR-ARD with more expressive models, e.g. **Bayesian neural networks**.
- Train per-sector "expert" models and **aggregate** them, e.g. via **distributed Gaussian process** models (product-of-experts).

The next chapter concludes the book with a broader discussion of future research directions in Bayesian quantitative finance.